DESENVOLVIMENTO BACK-END · UNIDADE IV

Persistência de Dados e ORM

Dando memória durável à API acadêmica

SGBD · SQL · ORM · Migrations · Índices · Transações

🎯 Objetivos de aprendizagem

🗄️

Compreender

o papel do SGBD, dos drivers, das conexões e dos pools

🛡️

Explicar

por que prepared statements tornam as consultas seguras

🔁

Diferenciar

SQL direto e ORM, Active Record e Data Mapper

🔗

Identificar

chaves e relacionamentos 1:1, 1:N e N:N

🗂️

Relacionar

migrations, seeders e índices à evolução do banco

🔒

Analisar

transações ACID e problemas de concorrência

💡 E se a API esquecesse tudo?

Imagine reiniciar a API acadêmica e descobrir que **todos os alunos, cursos e matrículas sumiram**.

- Dados só na memória **morrem** com o processo
- **Persistir** é armazenar de forma **durável**
- Mas persistir **não é só "salvar"**: os dados precisam continuar **corretos**

Analogia

Anotar recados num quadro branco funciona... até alguém apagar o quadro. O banco de dados é o **caderno de registros** que sobrevive a falhas e reinicializações.

Persistência = **durabilidade** + **correção** sob concorrência, falhas e mudanças

🗄️ SGBD, driver e conexão



🗄️

SGBD

Processo separado que administra o banco e entende **SQL**

🔌

Driver

Traduz as chamadas da linguagem para o **protocolo** do banco

♻️

Pool

Abrir conexão é **caro**: reutilizamos as que já existem

🔐 Em produção: conexão **cifrada** (`sslmode=require`) e um usuário de banco **só para a aplicação**, com o mínimo de permissões.

🛡️ Consultas seguras: prepared statements

```php
// ❌ Vulnerável: o valor digitado vira PARTE do comando SQL
$sql = "SELECT id, ra FROM alunos WHERE curso_id = " . $_GET['curso'];

// ✅ Seguro: o comando e o valor viajam SEPARADOS até o banco
$stmt = $pdo->prepare('SELECT id, ra FROM alunos WHERE curso_id = :curso');
$stmt->execute(['curso' => (int) $_GET['curso']]);   // valor tratado só como dado

// No Laravel, a fachada DB e o Query Builder também usam parâmetros
$alunos = DB::select('SELECT id, ra FROM alunos WHERE curso_id = ?', [$cursoId]);
```

📌 **O que demonstra:** no prepared statement o banco compila o comando com **marcadores** (`:curso` , `?`) e só depois recebe os valores. Eles **nunca** são interpretados como código SQL.

🗒️ SQL essencial para o back-end

Categoria	Finalidade	Comandos
DDL	definir a estrutura	`CREATE`, `ALTER`, `DROP`
DML	inserir, alterar e excluir	`INSERT`, `UPDATE`, `DELETE`
DQL	consultar dados	`SELECT` com `JOIN`, `WHERE`
DCL	conceder permissões	`GRANT`, `REVOKE`
TCL	controlar transações	`BEGIN`, `COMMIT`, `ROLLBACK`

💡 O boletim do aluno é um `SELECT` com **INNER JOIN** entre `matriculas`, `alunos` e `disciplinas`. Frameworks geram SQL, mas **é o SQL que chega ao banco**, e é lendo-o que encontramos lentidões e erros.

🔁 ORM: traduzindo objetos em tabelas

Código **orientado a objetos** e banco **relacional** falam línguas diferentes: o **descompasso de impedância**.

No código (objetos)	No banco (relacional)
Classe `Aluno`	Tabela `alunos`
Objeto	Linha
Atributo `ra`	Coluna `ra`
`$aluno->curso`	Chave estrangeira `curso_id`

Analogia

O ORM é um **intérprete**: você fala em objetos, ele traduz para SQL e devolve a resposta como objetos.

No Laravel, o ORM é o **Eloquent**: `$aluno->curso->nome` navega entre tabelas sem SQL manual.

⚖️ Active Record × Data Mapper

Critério	Active Record (Eloquent)	Data Mapper (Doctrine)
Quem grava os dados	a própria entidade: `$curso->save()`	um componente separado: `EntityManager`
Acoplamento ao ORM	**alto**: herda de `Model`	**baixo**: classe comum
Produtividade inicial	**alta**	média
Domínios complexos	média	**alta**
Onde aparece	Laravel, Ruby on Rails	Symfony, Hibernate, Entity Framework

Na API acadêmica, o melhor dos dois mundos: **Eloquent** pela produtividade, escondido atrás de **interfaces de repositório**.

🐢 A armadilha do N+1

```php
// ❌ N+1: 1 consulta para os alunos + 1 consulta POR aluno para o curso
foreach (Aluno::all() as $aluno) {
    echo $aluno->curso->nome;          // 100 alunos → 101 consultas
}

// ✅ Eager loading: os cursos são carregados antecipadamente
foreach (Aluno::with('curso')->get() as $aluno) {
    echo $aluno->curso->nome;          // 100 alunos → apenas 2 consultas
}
```

📌 **O que demonstra:** o ORM esconde o SQL, e por isso pode gerar **muitas consultas** sem você perceber.

🛠️ Em desenvolvimento, ative `Model::preventLazyLoading()` para o Laravel **acusar** o problema.

🔗 Chaves e relacionamentos

👤

1:1 · Um para um

Cada **usuário** tem, no máximo, um cadastro de **aluno**

📚

1:N · Um para muitos

Um **curso** possui várias **disciplinas**

🔀

N:N · Muitos para muitos

Alunos e **disciplinas**, ligados pela tabela `matriculas`

PK (chave primária): identifica cada linha de forma única, como o `id` do aluno.

FK (chave estrangeira): guarda a PK de outra tabela e garante a **integridade referencial**.

Quando a tabela intermediária tem **dados próprios** (semestre, status, nota), ela vira uma **entidade do negócio**: `Matricula`.

🗂️ Migrations: esquema como código

Arquivo
up() e down()

→

Git
versionado

→

`php artisan migrate`
só as pendentes

→

Banco igual em todos os ambientes

- **Migrations** registram cada mudança de estrutura
- **Seeders** criam dados iniciais (papéis, cursos)
- **Factories** geram dados fictícios em volume

> ⚠️ **Boa prática:** nunca altere uma migration já executada em outro ambiente. Crie uma **nova**, como `add_vagas_to_disciplinas`.

Um comando recria o banco do zero, igual ao dos colegas: **php artisan migrate:fresh --seed**

📖 Índices e desempenho

Situação	Recomendação
Colunas em `WHERE` e `JOIN` frequentes	criar índice
Chaves estrangeiras	criar índice
Colunas únicas (e-mail, RA)	índice **único**
Tabelas pequenas	geralmente desnecessário

Analogia

O **índice remissivo** de um livro aponta direto para a página certa. Sem ele, o banco lê a tabela inteira: o *full table scan*.

⚖️ Índices **não são gratuitos**: ocupam espaço e deixam as escritas mais lentas. Confirme sempre com `EXPLAIN` se a consulta usa o índice.

🗑️ Exclusão lógica (soft delete)

❌ Exclusão física

- `DELETE` remove a linha **para sempre**
- Perde o **histórico**
- Exclusão acidental **sem volta**

✅ Exclusão lógica

- Preenche a coluna `deleted_at`
- Consultas **ignoram** os excluídos
- Permite **auditar** e **restaurar**

⚠️ **Atenção:** restrições `unique` continuam valendo para os "excluídos". E a **LGPD** pode exigir a eliminação ou anonimização real de dados pessoais: aí o soft delete não basta.

🔒 Transações e propriedades ACID



Propriedade	Significado	Na matrícula
Atomicidade	tudo ou nada	matrícula e vaga gravadas juntas
Consistência	estado válido → estado válido	vagas nunca ficam negativas
Isolamento	transações não interferem	dois alunos não ocupam a última vaga
Durabilidade	após o commit, sobrevive a falhas	a matrícula não se perde no reinício

🏁 Concorrência: 40 vagas, 43 matriculados

```php
public function matricular(MatricularAlunoDTO $dto): Matricula
{
    return DB::transaction(function () use ($dto) {          // BEGIN
        $disciplina = Disciplina::whereKey($dto->disciplinaId)
            ->lockForUpdate()                                // SELECT ... FOR UPDATE
            ->firstOrFail();

        if ($disciplina->vagas <= 0) {
            throw new VagasEsgotadasException($disciplina->nome);   // → ROLLBACK
        }

        $matricula = Matricula::create([...]);               // INSERT
        $disciplina->decrement('vagas');                     // UPDATE vagas - 1
        return $matricula;                                   // → COMMIT
    });
}
```

📌 **O que demonstra:** a **transação** garante o tudo ou nada, e o **bloqueio pessimista** faz a segunda requisição **esperar** a primeira terminar, evitando a condição de corrida.

🚦 Isolamento e bloqueios

Nível	Leitura suja	Não repetível	Fantasma
Read Uncommitted	possível	possível	possível
Read Committed (PostgreSQL)	evitada	possível	possível
Repeatable Read (MySQL)	evitada	evitada	possível*
Serializable	evitada	evitada	evitada

🔒 Pessimista

Bloqueia a linha **antes** do conflito. Ideal quando a disputa é **alta**: matrículas.

🔢 Otimista

Cada registro tem uma **versão**; se ela mudou, a atualização falha com **409 Conflict**.

⚠️ Nada de e-mail ou API externa **dentro** da transação: use `DB::afterCommit()` ou filas.

🧭 Persistência no dia a dia: analogias

🛒 Transação · Tudo ou nada

Numa **compra online**, o pagamento e a baixa no estoque acontecem juntos. Se o pagamento falha, o produto volta para a prateleira.

Laravel: `DB::transaction()`

🚪 Bloqueio pessimista

O **provador da loja**: quem entra tranca a porta, e o próximo espera do lado de fora.

Laravel: `lockForUpdate()`

📝 Bloqueio otimista

O editor de texto avisa: **"este arquivo foi alterado por outra pessoa"**. Você recarrega e decide o que fazer.

Coluna `versao` + 409 Conflict

🚗 Deadlock · Impasse

Dois carros numa **ponte estreita**, cada um esperando o outro recuar. O banco cancela um deles.

Laravel: `attempts: 3`

💡 Pergunta-chave: **se duas pessoas fizerem isso ao mesmo tempo, o que pode dar errado?**

🌍 Aplicação em sistemas reais

🎟️

Venda de ingressos

Milhares disputando os mesmos assentos: **transação + bloqueio** evitam vender o mesmo lugar duas vezes

🏦

Bancos digitais

Débito numa conta e crédito na outra: **atomicidade** é obrigatória

🛍️

E-commerce

Catálogo com milhões de produtos: **índices** e **eager loading** mantêm a busca rápida

Operação de negócio → **Várias escritas?** → **Concorrência alta?** → **Transação + bloqueio adequado**

🧾 Síntese dos conceitos

- **SGBD**, **driver** e **pool** ligam a API ao banco
- **Prepared statements**: SQL e valores **separados**
- **ORM** traduz objetos ↔ tabelas, mas cuidado com o **N+1**
- **Active Record** (produtividade) × **Data Mapper** (isolamento)
- **PK**, **FK** e relacionamentos **1:1, 1:N e N:N**
- **Migrations** versionam o esquema; **índices** aceleram
- **Transações ACID** garantem o tudo ou nada
- **Bloqueios** protegem contra condições de corrida

🧭 O ORM é uma ferramenta poderosa, mas o **domínio de SQL e da modelagem relacional** é o que diferencia um bom profissional.

📝 Reflexão

```php
// Desafio: identifique os problemas deste código
// e diga como os conceitos da aula resolveriam cada um
$disciplina = Disciplina::find($id);

if ($disciplina->vagas > 0) {
    Matricula::create([...]);
    Mail::to($aluno)->send(new Confirmacao());
    $disciplina->decrement('vagas');
}
```

💬 Reflita

1. Por que uma regra como `unique` deve existir **também no banco**, e não só no código?
2. Quando você escolheria **soft delete** e quando ele **não** é suficiente?
3. Em que situação um **índice** pode **piorar** o desempenho?